

SIMULACIÓN DE ENTORNO: ASTRA DIVINITY V2.0

TIEMPO ESTIMADO DE COMPILACIÓN: 3.5 A 4 HORAS. NIVEL: SENIOR DBA.

Has solicitado carga de trabajo extrema. Este es el esquema físico transaccional desplegado en el servidor. Todas las consultas deben escribirse con sintaxis SQL estricta para PostgreSQL. No se aceptan pseudocódigos.

ESQUEMA FÍSICO (DDL BASE)

1. Tabla clientes (Proyección: 2 Millones de filas)

```
id_cliente SERIAL PK
nombre VARCHAR(100)
provincia VARCHAR(50)
telefono_emergencia_cifrado BYTEA
pin_entrega_hash TEXT
```

2. Tabla articulos (Proyección: 5,000 filas)

```
id_articulo SERIAL PK
nombre VARCHAR(100)
categoria VARCHAR(50)
stock INT
precio_unitario DECIMAL(10,2)
```

3. Tabla pedidos (Proyección: 15 Millones de filas)

```
id_pedido SERIAL PK
id_cliente INT FK (clientes)
id_articulo INT FK (articulos)
fecha DATE
cantidad INT
importe DECIMAL(12,2) CHECK (importe >= 0)
estado VARCHAR(20) -- ('PENDIENTE', 'EN_CAMINO', 'ENTREGADO', 'CANCELADO')
```

MÓDULO 1: DESPLIEGUE CRIPTOGRÁFICO Y RBAC

Task 1.1: Inyección de pgcrypto

El backend de Astra Divinity (Python/Flask) recibe el cliente `id_cliente = 150`. El teléfono crudo es `'291-555-0000'` y el PIN crudo es `'9911'`. Escribe la sentencia `UPDATE` que encripte simétricamente el teléfono (clave en RAM: `'Astra_Key_26'`) y hashee el PIN con `gen_salt` de blowfish y un costo de 10.

Task 1.2: Aislamiento de Capas

Crea un rol plantilla rol_logistica que NO pueda iniciar sesión. Luego, otórgale permiso de SELECT sobre pedidos, pero restríngale el UPDATE EXCLUSIVAMENTE a la columna estado. Finalmente, asígñale este rol al usuario operario_deposito.

MÓDULO 2: INGENIERÍA DE ÍNDICES Y CBO

Task 2.1: Profiling de Cuellos de Botella

El dashboard carga lento. La consulta es:

```
SELECT c.nombre, p.importe FROM clientes c JOIN pedidos p ON c.id_cliente =  
p.id_cliente WHERE p.estado = 'PENDIENTE' AND EXTRACT(MONTH FROM p.fecha) = 5 AND  
EXTRACT(YEAR FROM p.fecha) = 2026;
```

Reescribe la consulta para restablecer la SARGability. Luego, escribe el comando CREATE INDEX exacto (índice compuesto o de cobertura) en la tabla pedidos que resuelva esto con un Index-Only Scan si es posible, o un Bitmap Heap Scan altamente selectivo.

Task 2.2: Refactorización de Arquitectura

Justifica técnicamente en el siguiente cuadro por qué un índice en pedidos(estado) es probablemente ignorado por el motor (Full Table Scan) si el 90% de los pedidos históricos ya están 'ENTREGADO' y buscamos ese estado.

MÓDULO 3: VISTAS Y ESTADOS MATERIALIZADOS

Task 3.1: Security Definer

El rol_gerente necesita ver la suma de importe agrupada por categoria de artículo, pero NO tiene permisos sobre las tablas pedidos ni articulos. Escribe la CREATE VIEW estándar y los GRANT necesarios para tender este puente seguro.

Task 3.2: Snapshots Asíncronos

Crea una MATERIALIZED VIEW llamada v_ventas_historicas. Luego, escribe la estructura DDL (índice) necesaria y el comando de refresco exacto para actualizar la vista en producción sin inyectar un Access Exclusive Lock que congele las lecturas.

MÓDULO 4: CONCURRENCIA, ACID Y PURGAS

Task 4.1: Regla de Oro (Prevención de Deadlocks)

Un script de Python debe cancelar el `id_pedido = 800` y el `id_pedido = 120` simultáneamente. Escribe el bloque transaccional completo (`BEGIN; ... COMMIT;`) utilizando `SELECT ... FOR UPDATE` en el orden físico estricto para evitar colisiones de Row-Exclusive Locks en la caché.

Task 4.2: Punteros Huérfanos

El `operario_deposito` ha sido comprometido. Escribe el script de baja de 3 líneas que purga la matriz de acceso discrecional (DAC) antes de eliminar su identidad lógica del servidor.